A Beginner's Guide to Kafka® Consumers

August 25, 2022 | By **Ben Bromhead**

OPTIMIZE YOUR KAFKA PERFORMANCE TODAY!
Connect with our experts for tailored insights and solutions
Book free consultation

This guide will fill in all the gaps and explain what Kafka Consumers are, what Kafka Consumer Groups do, how to configure a Kafka Consumer Group, and show you how to use the Kafka console consumer while understanding key topics like what a Kafka offset is (and how to reset your Kafka offset).

Role of a Kafka Consumer

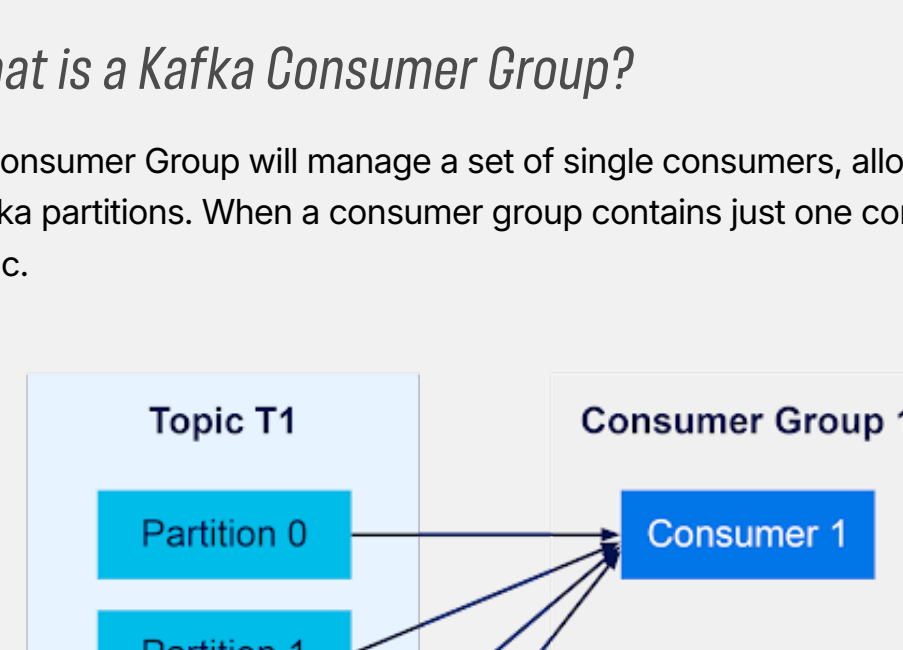
Apache Kafka® is made up of 2 types of external clients that interact with it: Producers send messages (they produce data) and Consumers retrieve data (they consume messages). Applications or services that need to receive messages will use a consumer to receive messages from topics within Apache Kafka. A Consumer is also generally the name of the class, object or function that is defined in the Kafka Driver that you will interact with when working with the Kafka driver API.

What is a Kafka Consumer?

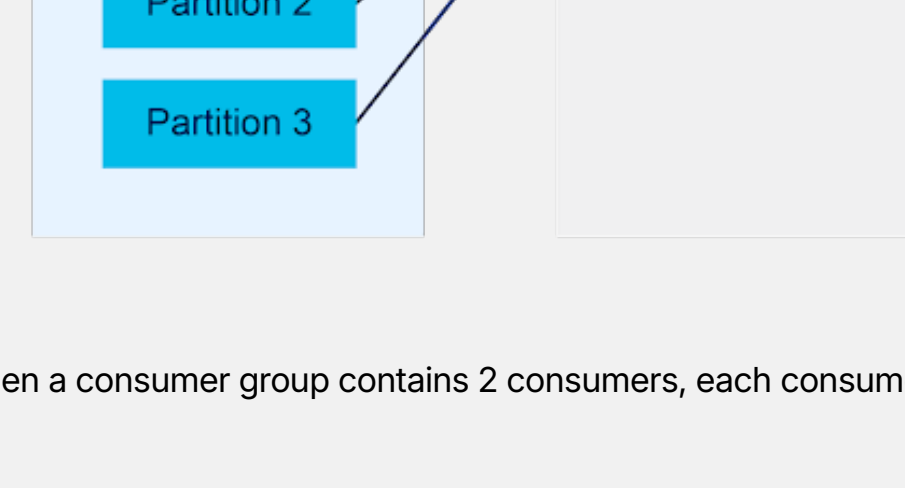
A single consumer will read messages from a number of Kafka topics , generally a consumer will be configured in a group of consumers, however you can configure a single consumer to read all messages from a single topic. In this case the consumer assigns all partitions within the topic to itself and receives all messages irrespective of the Kafka partitions. This approach is simple and may fit your use case, but misses out on the ability to load balance your consumers.

What is a Kafka Consumer Group?

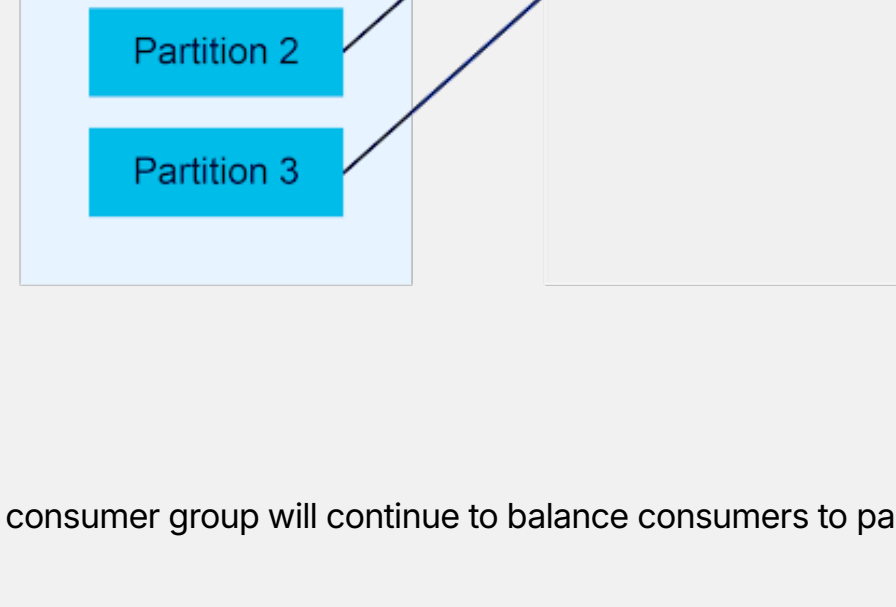
A Consumer Group will manage a set of single consumers, allowing Kafka and the consumers to distribute messages based on Kafka partitions. When a consumer group contains just one consumer, it will get all messages from all partitions in the Kafka topic.



When a consumer group contains 2 consumers, each consumer in the group will be assigned to half the partitions in the topic.



The consumer group will continue to balance consumers to partitions until the number of consumers is the same as partitions.



Once there are more consumers than partitions, the excess consumers will sit idle and receive no messages. This means the number of partitions in a topic will be the limit on the effective number of consumers that consume messages at any given time.

You can also have multiple Consumer Groups reading from the same topic. Each Consumer Group will maintain its own set of offsets and receive messages independently from other consumer groups on the same topic. To say it simply, a message received by Consumer Group 1 will also be received by Consumer Group 2. Within a single consumer group, a consumer will not receive the same messages as other consumers in the same consumer group.

What Is a Kafka Consumer Group ID?

Each consumer group is identified by a group id. It must be unique for each group, that is, two consumers that have different group ids will not be in the same group.

What Is a Kafka Consumer Offset and Commit?

After you start to read a bit more about Kafka Consumers and start configuring your own consumers, you will see the term "offset" and "commit" more and more. Apache Kafka does not have an explicit way of tracking which message has been read by a consumer of a group. Instead, it allows consumers to track the offset (the position in the queue) of what messages it has read for a given partition in a topic. To do this, a consumer in a consumer group will publish a special message topic for that topic/partition with the committed offset for each partition it has gotten up to.

Where Does Kafka Store Consumer Offsets?

Apache Kafka stores consumer offsets in a special internal topic called `__consumer_offsets`.

What Happens When a Consumer Joins or Leaves a Consumer Group?

When a new consumer is added to a consumer group, it will start consuming messages from partitions previously assigned to another consumer. It will generally pick up from the most recently committed offset. If a consumer leaves or crashes, the partitions it used to consume will be picked up by one of the consumers that is still in the consumer group. This change in partitions can also occur when a topic gets modified or more partitions are added to the topic.

The process of changing which consumer is assigned to what partition is called rebalancing. Rebalances are a normal part of Apache Kafka operations and will occur during configuration changes, scaling, or if a broker or consumer crashes. Generally, you want to avoid a rebalance if you care about real-time consumption of messages, as some messages cannot be read during a rebalance, which results in a loss of availability. This behavior has improved since Kafka 2.4 with the implementation of cooperative rebalancing which reduces, but does not totally eliminate some of this behaviour.

How Do Kafka Consumer Groups Work Internally?

Consumers in a group share ownership of a topic based on the partitions within that topic. Consumers are considered to be members of a consumer group and membership is maintained by a heartbeat mechanism. Consumers are considered to be part of the consumer group if they continue to send heartbeats (you guessed it, to a special Kafka topic) to a Kafka broker designated as the group coordinator. Brokers are different for different consumer groups.

If the Group Coordinator does not see a heartbeat from a consumer within a certain amount of time, it will consider the consumer to be dead and will start a rebalance. During the time from when a coordinator has seen the last heartbeat and when it marks a consumer as dead, messages for that partition are likely to build up as they are not being processed. If a consumer is shutdown cleanly it will notify a coordinator that it is leaving and minimize this window of unprocessed messages.

Note: The Group Coordinator will also mark a consumer as dead if it does not get polled by that consumer within a configurable period of time (e.g., 300 seconds). Keep this in mind if you are seeing rebalances despite successful heartbeats being sent (this may indicate your processing loop in your consumer application is stuck).

How Does the Group Coordinator Assign Partitions to a Kafka Consumer?

When a Kafka consumer first joins a Consumer Group it will receive a list of assigned partitions from the Group Leader. The Group leader is the first consumer to send a *JoinGroup* request to the Group Coordinator. The Group leader will maintain the full list of partition assignments. Consumers that are not the Group Leader will only see a list of their assigned partitions. Once the Group Leader has assigned partitions it sends it to the Group Coordinator which distributes the information to all consumers (Group Leaders do not communicate directly with other consumers). This process of assigning a Group Leader and assigning partitions happens every time a rebalance occurs.

Is Kafka Push or Pull?

Kafka Consumers pull data from the topic they are listening to. This allows different consumers to consume messages at different rates without the broker (Kafka) having to determine the right data rate for each consumer. It also allows better batch processing mechanics. Luckily the Kafka Consumer API allows client applications to treat the semantics as push if you wish to (e.g., you get a message as soon as it's ready) without the need to worry about the client getting overwhelmed. You do however need to keep an eye on your offset lag. Read https://www.instaclustr.com/blog/monitor-the-lag-in-your-real-time-applications-with-consumer-group-metrics/ for more details on offset lag and how Instaclustr can help manage it!

Kafka Concepts Cheat Sheet

We've put together this helpful graphic to illustrate the relationship between topics, consumers, partitions, and other concepts like replicas, brokers, producers etc. To learn more about these other concepts, checkout https://www.instaclustr.com/education/apache-kafka/apache-kafka-architecture-a-complete-guide-2026/

How to Configure Kafka Consumers

Getting Started

The process for setting up and creating a Kafka Consumer will be slightly different depending on your chosen development language. However generally the concepts will be the same. You will first create/instantiate a Kafka Consumer with its appropriate configuration e.g., group id, prior offset etc. Second, you will create some sort of loop that will process and handle the messages as they are received from the Kafka topic. Below is a basic example of how to create a Kafka Consumer in Java.

First create a properties file (you can define these properties programmatically if you wish, but using a properties file is generally going to be best).

We've also included some common authentication properties used in the Instaclustr Kafka platform (you can try it yourself! https://console2.instaclustr.com/signup).

You will also need to replace the text MYKAFKAIPADDRESS1 with your Kafka brokers IP addresses.

```
bootstrap.servers=MYKAFKAIPADDRESS1:9092,MYKAFKAIPADDRESS2:9092,MYKAFKAIPADDRESS3:9092
key.deserializer=org.apache.kafka.common.serialization.StringDeserializer
value.deserializer=org.apache.kafka.common.serialization.StringDeserializer
group.id=my-group
security.protocol=SASL_PLAINTEXT
sasl.mechanism=SCRAM-SHA-256
sasl.jaas.config=org.apache.kafka.common.security.scram.ScramLoginModule required \
    username="[USER NAME]" \
    password="[USER PASSWORD]";
```

Next create your Consumer. For this example, we've also included the program entry point (main) and a loop to process the messages.

```
import org.apache.kafka.clients.consumer.KafkaConsumer;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.clients.consumer.ConsumerRecords;

import java.io.FileReader;
import java.io.IOException;
import java.time.Duration;
import java.util.Collections;
import java.util.Properties;

public class Consumer {
    public static void main(String[] args) {

        Properties kafkaProps = new Properties();

        try (FileReader fileReader = new FileReader("consumer.properties")) {
            kafkaProps.load(fileReader);
        } catch (IOException e) {
            e.printStackTrace();
        }

        try (KafkaConsumer<String, String> consumer = new KafkaConsumer<>(kafkaProps)) {
            consumer.subscribe(Collections.singleton("test"));

            while (true) {
                ConsumerRecords<String, String> records = consumer.poll(Duration.ofMillis(100));
                for (ConsumerRecord<String, String> record : records) {
                    System.out.println(String.format("topic = %s, partition = %s, offset = %d, key = %s, value = %s",
                            record.topic(), record.partition(), record.offset(), record.key(), record.value()));
                }
            }
        }
    }
}
```

Configuration Options

There are a variety of configuration options for your Kafka Consumers. We've highlighted and explained some of the most common options for Kafka Consumers.

For all configuration options and to read more details, check out your Kafka Drivers documentation!

- **client.id** This will identify the client to the brokers in the Kafka cluster. It is used to correlate requests to the client that made them. It is generally recommended that all consumers in the same group have the same client ID to enforce client quotas for a single group.
- **session.timeout.ms** Default of 10 seconds. This value specifies the amount of time which the broker needs to get at least one heartbeat from the client before it marks the consumer as dead.
- **heartbeat.interval.ms** Default of 3 seconds. This value specifies the frequency in which a consumer will send a heartbeat signal.
- **max.poll.interval.ms** Default 300 seconds. Specifies how long a broker will wait between calls to the poll method causing the consumer to try fetching more messages before it marks a consumer as dead.
- **enable.auto.commit** Default enabled. The consumer will automatically commit offsets periodically at the interval set by auto.commit.interval.ms which is 5 seconds by default.
- **fetch.min.bytes** The minimum amount of data a consumer will fetch from a broker. If the broker has less data available than this value, the consumer will wait until more is available. This can be used to minimize the backwards and forwards between consumers and brokers (potentially improving throughput) at the expense of latency).
- **fetch.max.wait.ms** The maximum amount of time to wait before fetching messages from the broker. Used in conjunction with fetch.min.bytes this can set an upper bound on how long a consumer will wait. The consumer will fetch messages from the broker if this value is exceeded
- **Max.partition.fetch.bytes** The maximum amount of bytes to fetch on a per partition basis. Useful for limiting the upper bound amount of memory a client requires for fetching messages. Keep in mind the max.poll.interval settings as well when setting this value as a large value of this may result in more data delivered than is possible to fetch in the poll interval window.
- **auto.offset.reset** Controls how a consumer will behave when it tries reading a partition that doesn't have a previous last read offset. There are two settings: latest and earliest. Latest will start from the latest available message and earliest will start from the earliest available offset.
- **partition.assignment.strategy** The partition assignment strategy controls how the Group Leader consumer will divide up partitions between consumers in the consumer group.
- **max.poll.records** Maximum number of records to be fetched in a single poll call. Useful to control throughput within each consumer.

What Is the Difference Between session.timeout.ms and heartbeat.interval.ms?

As described above, the session timeout is how long a broker will wait for a heartbeat before marking a consumer as dead, whereas the heartbeat interval is how often a consumer sends a heartbeat. Generally, you want the client to send multiple heartbeats within the session timeout window. If you just use the defaults, you will be fine (10 second timeout, with a 3 second heartbeat).

Summary

Apache Kafka offers a uniquely versatile and powerful architecture for streaming workloads with extreme scalability, reliability, and performance. With this guide you should be ready to start building your own Kafka consumer and understand how it works under the hood!

To learn more about how Instaclustr's Managed Services can help your organization make the most of Kafka and all of the 100% open source technologies available on the Instaclustr Managed Platform, sign up for a free trial here or see the pricing for Instaclustr managed services here.

Master Apache Kafka with expert insights
Struggling to strike the perfect balance between performance, scalability, and reliability in your Kafka ecosystem? Whether you're optimizing for zero downtime or maximizing resource efficiency, this guide has you covered.
Free download

Add Instaclustr as a preferred source on Google

Related content

A Visual Understanding to Ensuring Your Apache Kafka® Data is (Literally) in Order

A visual guide to help understand some rather complex Kafka data nuances when trying to store and retrieve data in your preferred ...

Apache Kafka® architecture: A complete guide [2026]

Apache Kafka is a distributed event streaming platform for building real-time data pipelines and streaming applications. It works ...

Spinning Apache Kafka® Microservices With Cadence Workflows - Part 2

In "Spinning your Workflows With Cadence!" we explored Cadence, a new scalable, developer-focused, open source workflow engine. ...

NetApp uses cookies
By clicking "Accept all", you agree to the storing of cookies on your device to enhance site navigation, analyze site usage, and assist in our marketing efforts. Certain essential cookies are required, but you can customize your preferences by selecting "Cookie settings".
Accept All Do not sell or share my information
Cookie settings See our Cookie Policy for more information.